

NEWS HEADLINE SARCASM DETECTION USING NLP AND MACHINE LEARNING

Mini NLP Project Report

Submitted in partial fulfillment of the requirements for the course

Natural Language Processing (245DSEUBC303B)

Submitted by

ANDREWS R KATTOOR

24UBC109

III BCA A

Marian College Kuttikkanam Autonomous

Academic Year: 2026–2027

Abstract

Sarcasm is a form of language in which the intended meaning may differ from the literal meaning of a statement. Detecting sarcasm from written text is a challenging Natural Language Processing (NLP) task because the meaning of a statement can depend on wording, context, and linguistic patterns.

This project presents a machine learning based system for detecting sarcasm in news headlines. The **News Headlines Dataset for Sarcasm Detection** was used for training and evaluation. The dataset contains 28,619 original headlines, which were reduced to 28,503 records after duplicate removal.

The proposed system performs basic text preprocessing, including lowercasing, URL removal, and whitespace normalization. The processed headlines are converted into numerical representations using **Term Frequency-Inverse Document Frequency (TF-IDF)** with unigram and bigram features. Three machine learning models — **Logistic Regression, Multinomial Naive Bayes, and Linear Support Vector Machine (SVM)** — were trained and compared.

Among the evaluated models, Linear SVM achieved the best overall performance with an **accuracy of 85.14%** and an **F1-score of 84.53%**. The selected model was integrated into a Streamlit-based Python application that allows users to enter a news headline and receive a sarcasm prediction.

The project also explores the potential use of sarcasm detection as an accessibility-oriented technology. The current news-headline application serves as a prototype for a future browser-based system capable of analysing online content and providing additional contextual assistance.

3. Introduction

News headlines are designed to communicate information quickly and attract the attention of readers. However, some headlines use **sarcasm**, where the intended meaning differs from the literal meaning of the words. Detecting sarcasm is challenging for computers because it often depends on context, wording, contrast, and implied meaning rather than on specific keywords alone.

Natural Language Processing (NLP) provides techniques for processing and analyzing human language so that computers can identify useful patterns in text. In this project, NLP and machine learning techniques are applied to classify news headlines as **sarcastic** or **non-sarcastic**.

The project uses the **News Headlines Dataset for Sarcasm Detection**, which contains news headlines labelled according to whether they are sarcastic. The text is cleaned using basic preprocessing techniques and converted into numerical features using **TF-IDF (Term Frequency–Inverse Document Frequency)**. Three machine learning algorithms—Logistic Regression, Multinomial Naive Bayes, and Linear Support Vector Machine (SVM)—are trained and compared.

The final Linear SVM model is integrated into a simple Streamlit application where a user can enter a news headline and receive a sarcasm prediction. The project also explores the potential use of sarcasm detection as an accessibility-oriented tool for users who may find implied meanings and sarcasm difficult to interpret.

4. Problem Statement

Sarcasm is commonly used in news headlines to create humour, irony, or emphasis. Although humans can often understand sarcastic expressions using context and language cues, automatically identifying sarcasm from text is a challenging NLP problem.

A system is therefore required to analyze a given news headline and classify it into one of two categories:

- **Sarcastic**
- **Non-Sarcastic**

The system should process the input headline, extract meaningful textual features, apply a trained machine learning classification model, and provide the predicted category to the user.

The proposed project addresses this problem by developing a lightweight NLP-based sarcasm detection system using **TF-IDF feature extraction and supervised machine learning classification**.

5. Objectives

The main objectives of this project are:

1. To study sarcasm detection as a **Natural Language Processing classification problem**.
2. To obtain and analyze a labelled dataset containing sarcastic and non-sarcastic news headlines.
3. To preprocess the textual data by performing operations such as lowercasing, URL removal, and whitespace normalization.
4. To convert textual headlines into numerical representations using **TF-IDF**.
5. To implement and compare multiple machine learning algorithms:
 - Logistic Regression
 - Multinomial Naive Bayes
 - Linear Support Vector Machine (SVM)
6. To evaluate the models using **accuracy, precision, recall, F1-score, and confusion matrix**.
7. To select the best-performing model for the final prediction system.
8. To develop a simple user interface that allows users to enter headlines and obtain sarcasm predictions.
9. To explore the potential of sarcasm detection as an accessibility-oriented NLP application.

6. Motivation and Accessibility Relevance

6.1 Motivation

The motivation for this project comes from the difficulty of understanding sarcasm and implied meanings in written communication. Unlike literal statements, sarcastic expressions often communicate an intended meaning that is different from the actual words used. This can make sarcastic content difficult to interpret automatically and, in some situations, difficult for people to understand.

The project was therefore selected as an application of NLP that addresses a practical language-understanding problem. News headlines were chosen because they are short, information-rich pieces of text and frequently use humour, irony, exaggeration, or unexpected wording to attract readers.

The project also provides an opportunity to understand the complete machine learning workflow, including:

- Dataset collection and analysis
- Text preprocessing
- Feature extraction
- Model training
- Model comparison
- Performance evaluation
- Development of a working prediction application

6.2 Accessibility Relevance

Sarcasm detection can also have potential accessibility applications. Some users may find sarcasm, irony, or implied meanings difficult to interpret from written communication. An automated system that identifies potentially sarcastic content could provide an additional layer of information and help users distinguish between literal and non-literal statements.

In this project, the news-headline classifier is treated as a **prototype for an accessibility-oriented NLP application** rather than as a solution to communication difficulties. The current system focuses specifically on news headlines, while a more advanced version could analyze text from different types of online content.

6.3 Future Accessibility Vision

The long-term vision of the project is to extend the system beyond a standalone application. A future version could be developed as a **browser extension or web-based tool** that analyzes online text in real time and highlights potentially sarcastic statements.

Such a system could potentially:

1. Detect sarcastic or potentially non-literal statements.
2. Provide a visual indication to the user.
3. Analyze content across different websites.
4. Allow users to enable or disable sarcasm detection according to their preference.
5. Support a more comfortable and accessible web-reading experience.

The current project therefore serves as a **proof-of-concept** for exploring how NLP and machine learning can be applied to improve the interpretation of written online content.

7. Dataset Description

The project uses the **News Headlines Dataset for Sarcasm Detection**, a labelled dataset containing news headlines classified as either sarcastic or non-sarcastic.

The dataset contains three main attributes:

Attribute	Description
<code>is_sarcastic</code>	Target label indicating whether the headline is sarcastic
<code>headline</code>	Text of the news headline
<code>article_link</code>	Link associated with the original article

The original dataset contained **28,619 records**. After removing duplicate records, the dataset used for modelling contained **28,503 records**.

7.1 Class Distribution

The dataset contains two classes:

Class	Number of Records	Percentage
Non-Sarcastic	14,985	52.36%
Sarcastic	13,634	47.64%
Total	28,619	100%

The two classes are relatively balanced, which is useful for training a binary classification model.

7.2 Data Cleaning

Duplicate records were removed before model training. The dataset was also checked for missing and duplicate headlines.

The cleaned dataset was then used for text preprocessing and feature extraction.

7.3 Train-Test Split

The cleaned data was divided into:

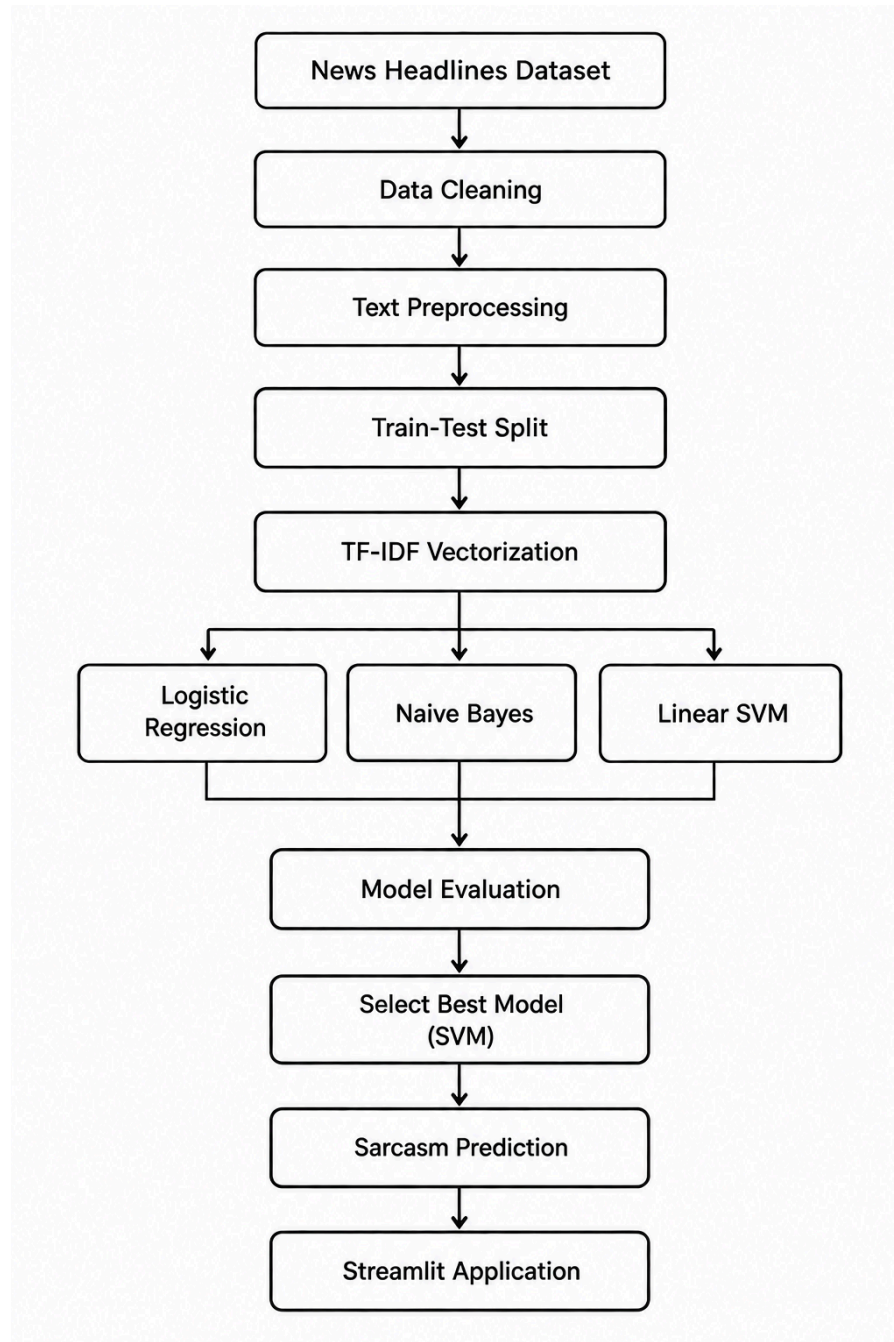
- **80% training data — 22,802 headlines**
- **20% testing data — 5,701 headlines**

A stratified split was used so that the proportion of sarcastic and non-sarcastic headlines remained approximately consistent between the training and testing sets.

8. Methodology

The proposed sarcasm detection system follows a sequence of steps that transforms raw news headlines into predictions. The overall methodology consists of **data preprocessing**, **dataset splitting**, **feature extraction**, **model training**, **model comparison**, **evaluation**, and **final prediction**.

8.1 Overall Workflow



8.2 Data Preprocessing

Raw textual data cannot be directly provided to most traditional machine learning algorithms. Therefore, the headlines were first cleaned using basic text preprocessing techniques.

The following operations were performed:

1. Lowercasing

All headlines were converted to lowercase so that words such as **Government** and **government** were treated as the same term.

2. URL Removal

URLs and web links were removed because they do not provide useful textual information for the classification task.

3. Whitespace Normalization

Extra spaces and unnecessary whitespace were removed to produce cleaner text.

The preprocessing approach was intentionally kept conservative. Punctuation, stopwords, and the original wording were not aggressively removed because such features can potentially contribute to identifying sarcasm.

8.3 Dataset Splitting

After preprocessing, the dataset was divided into training and testing sets using an **80:20 ratio**.

- Training set: **22,802 headlines**
- Testing set: **5,701 headlines**

A **random_state** of 42 was used to make the experiment reproducible. **Stratified sampling** was applied so that both classes maintained a similar distribution in the training and testing datasets.

8.4 TF-IDF Feature Extraction

Since machine learning algorithms require numerical input, the cleaned headlines were converted into numerical feature vectors using **TF-IDF (Term Frequency–Inverse Document Frequency)**.

TF-IDF assigns a numerical importance value to words based on:

- How frequently a word occurs in a particular headline.
- How common or uncommon the word is across the complete collection of headlines.

The project uses both **unigrams** and **bigrams**:

- **Unigram**: one word, e.g., **government**
- **Bigram**: two consecutive words, e.g., **government fails**

Using bigrams allows the model to capture short word combinations that may provide useful contextual information.

A minimum document frequency of 2 was used, meaning terms occurring in only one document were excluded.

The resulting TF-IDF representation contained **33,967 features**.

8.5 Machine Learning Models

Three supervised machine learning algorithms were implemented and compared.

8.5.1 Logistic Regression

Logistic Regression is a classification algorithm that estimates the probability of an input belonging to a particular class.

In this project, it was used as one of the baseline models for distinguishing between sarcastic and non-sarcastic headlines.

8.5.2 Multinomial Naive Bayes

Multinomial Naive Bayes is commonly used for text classification because it works well with numerical representations of word frequencies and text features.

It was included to provide a probabilistic approach for comparison with the other models.

8.5.3 Linear Support Vector Machine

Linear SVM attempts to find a decision boundary that separates the two classes while maximizing the margin between them.

It is particularly suitable for high-dimensional text representations such as TF-IDF. In this project, Linear SVM achieved the best overall performance among the three tested models and was therefore selected as the final classifier.

8.6 Model Evaluation

The models were evaluated using four main performance metrics:

Accuracy

Accuracy represents the proportion of correctly classified headlines among all test headlines.

Precision

Precision measures how many headlines predicted as sarcastic were actually sarcastic.

Recall

Recall measures how many of the actual sarcastic headlines were correctly identified by the model.

F1-Score

F1-score combines precision and recall into a single metric and is useful when evaluating classification performance using both types of errors.

A **confusion matrix** was also generated for the selected Linear SVM model to visualize correct and incorrect predictions.

8.7 Model Selection

After training the three models, their performance was compared using the evaluation metrics.

Model	Accuracy	Precision	Recall	F1-Score
Logistic Regression	84.74%	83.05%	85.32%	84.17%
Naive Bayes	84.90%	86.10%	81.37%	83.67%
Linear SVM	85.14%	83.72%	85.36%	84.53%

Linear SVM was selected as the final model because it achieved the **highest accuracy and F1-score** among the three tested approaches.

8.8 Final Prediction Process

Once the Linear SVM model was selected, the trained model and TF-IDF vectorizer were saved using `joblib`.

For a new headline, the application performs the following steps:

User enters headline



Preprocess headline



Convert to TF-IDF vector



Pass vector to Linear SVM



Generate prediction



Sarcastic / Non-Sarcastic

This trained model is then used by the Streamlit application to provide predictions for new headlines.

9. Implementation

The sarcasm detection system was implemented using **Python**. The implementation consists of two main components:

- 1. **Model training and evaluation**
- 2. **Streamlit-based prediction application**

The training program prepares the dataset, performs preprocessing, extracts TF-IDF features, trains multiple machine learning models, evaluates their performance, and saves the selected model. The Streamlit application loads the saved model and provides a simple interface for predicting sarcasm in new headlines.

9.1 Tools and Technologies Used

Technology	Purpose
Python	Main programming language
Pandas	Dataset loading and manipulation
Regular Expressions	Text preprocessing
Scikit-learn	TF-IDF, machine learning and evaluation
Joblib	Saving and loading trained models
Matplotlib	Confusion matrix visualization
Streamlit	User interface for the prediction system

9.2 Dataset Loading

The dataset is stored in JSON format. Each line represents a news headline and its corresponding information.

The dataset was loaded using Python's `json` module and converted into a Pandas DataFrame for further processing.

```
data = []
```

```
with open(DATASET_PATH, "r", encoding="utf-8") as file:
```

```
    for line in file:
```

```
        data.append(json.loads(line))
```

```
data = pd.DataFrame(data)
```

After loading, the dataset was checked and duplicate records were removed.

```
data = data.drop_duplicates()
```

```
data = data.drop_duplicates(
```

```
    subset=["headline", "is_sarcastic"]
```

```
)
```

9.3 Text Preprocessing

A preprocessing function was created to clean the headline text.

```
def preprocess_text(text):
```

```
    text = text.lower()
```

```
    text = re.sub(r"http\S+|www\S+", "", text)
```

```
    text = re.sub(r"\s+", " ", text).strip()
```

```
    return text
```

The function was applied to every headline:


```
data["clean_headline"] = data["headline"].apply(preprocess_text)
```

This produced a cleaned version of each headline that could be used for feature extraction.

9.4 Train-Test Split

The cleaned headlines and their labels were separated into input and target variables.

```
X = data["clean_headline"]
```

```
y = data["is_sarcastic"]
```

The data was then divided into training and testing sets.

```
X_train, X_test, y_train, y_test = train_test_split(
```

```
    X,
```

```
    y,
```

```
    test_size=0.20,
```

```
    random_state=42,
```

```
    stratify=y
```

```
)
```

The training set was used to train the models, while the testing set was kept for evaluating their performance.

9.5 TF-IDF Implementation

TF-IDF vectorization was performed using Scikit-learn.

```
tfidf = TfidfVectorizer(
```

```
    ngram_range=(1, 2),
```

```
    min_df=2
```

```
)
```

```
X_train_tfidf = tfidf.fit_transform(X_train)
```

```
X_test_tfidf = tfidf.transform(X_test)
```

The vectorizer was fitted **only on the training data** and then used to transform the testing data. This prevents information from the test set from influencing the feature extraction process.

The resulting training data contained **33,967 TF-IDF features**.

9.6 Model Training

Three classification algorithms were trained:

```
models = {
```

```
    "Logistic Regression":
```

```
        LogisticRegression(max_iter=1000),
```

```
    "Naive Bayes":
```

```
        MultinomialNB(),
```

```
    "Linear SVM":
```

```
        LinearSVC()
```

```
}
```

Each model was trained using the TF-IDF training features.

```
for name, model in models.items():
```

```
    model.fit(X_train_tfidf, y_train)
```

```
    predictions = model.predict(X_test_tfidf)
```

```
    accuracy = accuracy_score(y_test, predictions)
```

```
    precision = precision_score(y_test, predictions)
```

```
recall = recall_score(y_test, predictions)
```

```
f1 = f1_score(y_test, predictions)
```

The resulting performance values were stored and compared to determine the best-performing classifier.

9.7 Saving the Final Model

Linear SVM was selected as the final model based on its evaluation results.

Both the trained model and TF-IDF vectorizer were saved using Joblib:

```
joblib.dump(  
    svm_model,  
    "sarcasm_svm_model.pkl"  
)
```

```
joblib.dump(  
    tfidf,  
    "tfidf_vectorizer.pkl"  
)
```

These files allow the application to use the already-trained model without retraining it every time the application starts.

9.8 Confusion Matrix

A confusion matrix was generated for the final Linear SVM model.

```
ConfusionMatrixDisplay.from_predictions(  
    y_test,  
    svm_predictions,  
    display_labels=[
```

```
"Non-Sarcastic",  
"Sarcastic"  
]  
)
```

The resulting confusion matrix was saved as an image and used in the project report to visually represent the classification results.

9.9 Streamlit Application

A Streamlit application was developed to provide an interactive interface for the trained model.

At startup, the application loads the saved model and TF-IDF vectorizer:

```
model = joblib.load(  
    "sarcasm_svm_model.pkl"  
)
```

```
tfidf = joblib.load(  
    "tfidf_vectorizer.pkl"  
)
```

The user enters a news headline through a text input field. The headline is then:

1. Preprocessed.
2. Converted into a TF-IDF vector.
3. Passed to the Linear SVM model.
4. Classified as **Sarcastic** or **Non-Sarcastic**.

The prediction function is:

```
def predict_sarcasm(headline):  
    cleaned_text = preprocess_text(headline)  
    vector = tfidf.transform([cleaned_text])  
    prediction = model.predict(vector)[0]
```

```
if prediction == 1:  
    return "Sarcastic"  
  
else:  
    return "Non-Sarcastic"
```

The application provides the prediction directly to the user through the Streamlit interface.

9.10 Application Output

For testing, the application was provided with the headline:

"Government solves traffic problem by asking everyone to stay home"

The system predicted:

Sarcastic

This demonstrates the complete pipeline from user input to machine learning prediction.

The implementation satisfies the assignment requirement that the NLP application be implemented using **Python** and that complete, readable, organized, and executable source code be provided.

10. Results and Evaluation

The developed sarcasm detection system was evaluated using the test dataset containing **5,701 headlines**. Three machine learning models were trained and compared using accuracy, precision, recall, and F1-score.

10.1 Model Performance Comparison

The obtained results are shown below:

Model	Accuracy	Precision	Recall	F1-Score
Logistic Regression	84.74%	83.05%	85.32%	84.17%
Naive Bayes	84.90%	86.10%	81.37%	83.67%
Linear SVM	85.14%	83.72%	85.36%	84.53%

Among the three models, **Linear SVM achieved the highest accuracy of 85.14% and the highest F1-score of 84.53%**. It also achieved the highest recall at 85.36%.

Therefore, Linear SVM was selected as the final classification model for the application.

10.2 Interpretation of Model Results

Logistic Regression achieved an accuracy of 84.74%. Its recall of 85.32% indicates that it was able to identify a large proportion of the sarcastic headlines.

Naive Bayes achieved 84.90% accuracy and the highest precision of 86.10%. However, its recall was lower at 81.37%, meaning that it missed more sarcastic headlines compared with the other models.

Linear SVM achieved the best overall performance, with 85.14% accuracy and an F1-score of 84.53%. Its balanced precision and recall made it the most suitable model for the final application.

10.3 Confusion Matrix

The confusion matrix for the final Linear SVM model was:

Actual / Predicted	Non-Sarcastic	Sarcastic
	c	c
Non-Sarcastic	2540	450
Sarcastic	397	2314

The model correctly classified:

- **2,540** non-sarcastic headlines.
- **2,314** sarcastic headlines.

The model incorrectly classified:

- **450** non-sarcastic headlines as sarcastic.
- **397** sarcastic headlines as non-sarcastic.

Overall, the model correctly classified **4,854 out of 5,701 test headlines**, resulting in an accuracy of approximately **85.14%**.

10.4 Classification Report

The final Linear SVM classification report was:

Class	Precision	Recall	F1-Score	Support
Non-Sarcastic	0.86	0.85	0.86	2990
Sarcastic	0.84	0.85	0.85	2711
Accuracy	0.85	—	—	5701
Macro Avg	0.85	0.85	0.85	5701
Weighted Avg	0.85	0.85	0.85	5701

Accuracy: 85%

The results show that the model performs reasonably consistently for both classes. The F1-score is approximately **0.86 for non-sarcastic headlines** and **0.85 for sarcastic headlines**, indicating that neither class is being ignored by the classifier.

10.5 Sample Application Output

The trained model was integrated into the Streamlit application.

A sample headline used for testing was:

Government solves traffic problem by asking everyone to stay home

The application produced the following prediction:

Sarcastic

This demonstrates that the trained model can be used to classify previously unseen headline text through the application interface.

10.6 Overall Result

The experiment demonstrates that traditional machine learning combined with TF-IDF can effectively perform sarcasm classification on news headlines.

The final system achieved:

Evaluation Measure	Result
Accuracy	85.14%
Precision	83.72%
Recall	85.36%
F1-Score	84.53%

The results satisfy the assignment's recommendation to evaluate the NLP application using classification metrics and supporting outputs such as tables, graphs, confusion matrices, and sample results.

11. Application

The final trained model was deployed through a **Streamlit-based interactive application**. The application provides a simple interface through which a user can enter a news headline and obtain a predicted classification.

The application contains:

- A headline input field.
- A prediction button.
- Prediction result display.
- Model information.
- Accuracy and F1-score information.
- A short description of the project.

Application Workflow

User enters headline



Text preprocessing



TF-IDF transformation



Linear SVM classifier



Prediction



Sarcastic / Non-Sarcastic

The application makes the trained NLP model accessible without requiring the user to interact directly with Python code.

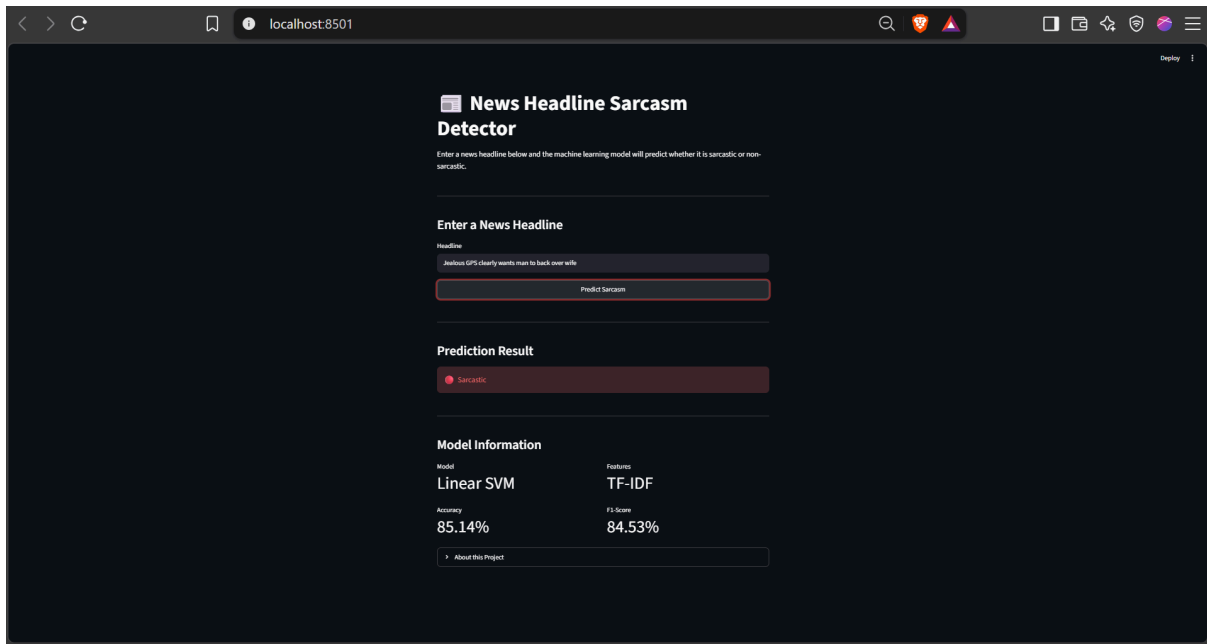


Figure 1: Streamlit application showing the sarcasm prediction.

```
C:\BCA\sem5\NLP\PROJECT\News-Headline-Sarcasm-Detection>python train_model.py
Original dataset shape: (28619, 3)
Dataset shape after removing duplicates: (28503, 3)
Training samples: 22802
Testing samples: 5701
Number of TF-IDF features: 33967

Logistic Regression
Accuracy : 84.74 %
Precision: 83.05 %
Recall : 85.32 %
F1 Score : 84.17 %

Naive Bayes
Accuracy : 84.9 %
Precision: 86.1 %
Recall : 81.37 %
F1 Score : 83.67 %

Linear SVM
Accuracy : 85.14 %
Precision: 83.72 %
Recall : 85.36 %
F1 Score : 84.53 %

Final model selected: Linear SVM

Model saved as: sarcasm_svm_model.pkl
Vectorizer saved as: tfidf_vectorizer.pkl

C:\BCA\sem5\NLP\PROJECT\News-Headline-Sarcasm-Detection>
```

Figure 2: Comparison of the three machine learning models.

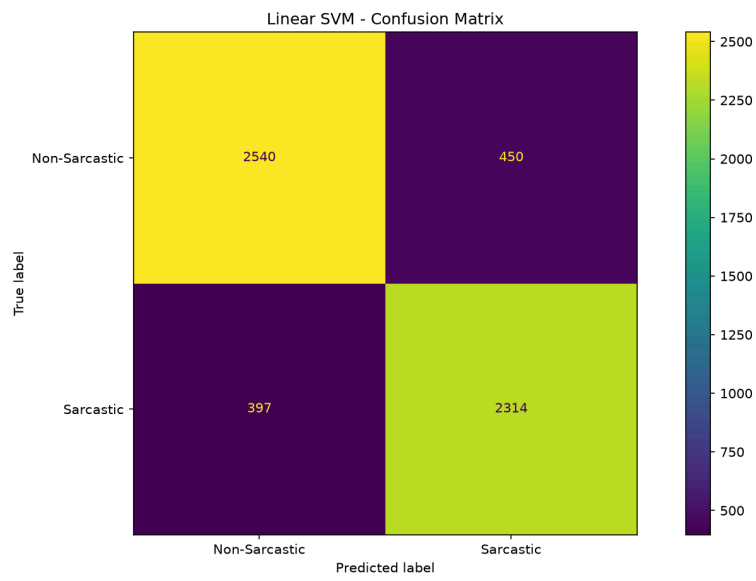


Figure 3: Confusion matrix of the Linear SVM model.

12. Limitations

Although the developed system achieved an accuracy of **85.14%**, several limitations remain.

12.1 Limited Context

The current system analyzes individual news headlines rather than the complete article or surrounding context. Sarcasm can sometimes depend on information that is not present in the headline itself.

12.2 Dataset-Specific Performance

The model was trained using a specific news-headline dataset. Therefore, its performance may decrease when it encounters headlines from different sources, writing styles, or domains.

12.3 Difficulty with Complex Sarcasm

Sarcasm can involve cultural references, world knowledge, humour, irony, or subtle linguistic patterns. TF-IDF-based features may not capture all of these aspects.

12.4 No Deep Semantic Understanding

The Linear SVM model works with numerical TF-IDF representations. It identifies statistical patterns in the text but does not possess human-like understanding of meaning, context, or intent.

12.5 Binary Classification

The system produces only two outcomes:

- Sarcastic
- Non-Sarcastic

It does not provide a sarcasm intensity score or explain why a particular headline was classified as sarcastic.

12.6 False Predictions

The confusion matrix shows that some headlines were incorrectly classified. In particular, **450 non-sarcastic headlines were predicted as sarcastic**, while **397 sarcastic headlines were predicted as non-sarcastic**.

Therefore, the system should be considered a prediction aid rather than a completely reliable sarcasm detector.

13. Future Scope

The current project provides a basic prototype that can be extended in several ways.

13.1 Browser Extension

The most significant future improvement would be developing the system as a **browser extension**. Instead of requiring users to manually enter headlines, the extension could analyze text appearing on webpages and identify potentially sarcastic statements.

13.2 Real-Time Web Content Analysis

A future system could process online content in real time and provide visual indicators when sarcasm is detected. This could make the system more useful for accessibility-oriented applications.

13.3 Context-Aware Detection

The system could be improved by considering the surrounding article or previous sentences rather than analysing only an individual headline. Additional context could help the model distinguish between literal and sarcastic statements.

13.4 Deep Learning and Transformer Models

Future versions could experiment with advanced NLP architectures such as:

- Recurrent Neural Networks (RNNs)
- LSTMs
- BERT
- Other transformer-based language models

These models can capture more complex relationships between words and their context than traditional TF-IDF-based approaches.

13.5 Multilingual Sarcasm Detection

The current system focuses on English headlines. A future version could support multiple languages, making the application useful to a wider range of users.

13.6 Explainable Predictions

The application could provide an explanation alongside the prediction, such as highlighting words or phrases that contributed strongly to the classification.

13.7 Improved User Customization

Future versions could allow users to customize how detected sarcasm is displayed, for example through labels, highlights, or optional explanations.

14. Conclusion

This project successfully developed a **News Headline Sarcasm Detection system using Natural Language Processing and Machine Learning**.

The system used the News Headlines Dataset for Sarcasm Detection and applied basic text preprocessing followed by **TF-IDF feature extraction**. Three machine learning algorithms—Logistic Regression, Multinomial Naive Bayes, and Linear SVM—were implemented and evaluated.

Among the tested models, **Linear SVM achieved the best overall performance**, obtaining an accuracy of **85.14%** and an F1-score of **84.53%**. A confusion matrix was also used to evaluate the model's correct and incorrect classifications.

The trained model was integrated into a **Streamlit application**, allowing users to enter a news headline and receive a prediction of whether it is sarcastic or non-sarcastic.

Overall, the project demonstrates how NLP and machine learning can be combined to address a practical text-classification problem. It also provides a foundation for future accessibility-oriented applications, particularly a browser-based system capable of identifying potentially sarcastic content while users browse online.

The assignment emphasizes that the project should demonstrate a **working NLP application**, include implementation and results, and discuss limitations and future improvements.

15. References

1. Misra, R. **News Headlines Dataset for Sarcasm Detection**. GitHub repository.
 2. Scikit-learn documentation — Machine Learning algorithms and TF-IDF feature extraction.
 3. Streamlit documentation — Python framework for developing data and machine learning applications.
 4. Python documentation — Python programming language and standard libraries.
-